ШИНЖЛЭХ УХААН, ТЕХНОЛОГИЙН ИХ СУРГУУЛЬ
МЭДЭЭЛЭЛ, ХОЛБООНЫ ТЕХНОЛОГИЙН СУРГУУЛЬ



Бие даалт-1
тайлан

Үйлдлийн систем (F.CSM302)

Төслийн нэр:	Multithreaded Sorting Application
Хуваарилалтын код:	V-1-6
Хичээл заасан багш:	Г.Ганбат /F.CS10/
Бие даалтын ажил гүйцэтгэсэн:	B210900026 О. Хишигжаргал
	B231910001 Л. Батзориг
	B241910003 Б. Эрдэнэжарга
	B241910026 Б.Заяабуян.
	B241940015 Б. Хүслэн
	B242270135 М. Мөнхүүжин

Улаанбаатар хот, 2026.

Гарчиг

Хураангуй 3
1. Удиртгал 4
1.1 Зорилго 4
1.2 Ач холбогдол 4
1.3 Хамрах хүрээ 4
2. Сурах бичгийн шаардлага 5
2.1 Шаардлагуудын хэрэгжилт…….
3. Онолын үндэс 6
3.1 Үйлдлийн системийн ойлголт…..... 6
3.2 Алгоритм 6
3.3 Өгөгдлийн бүтэц..... 7
4. Системийн зохиомж 9
5. Хэрэгжилт 11
5.1 Хэл, сан, орчин..... 11
5.2 Массив хуваах..... 11
5.3 Worker-ийн ажиллагаа..... 11
6. Туршилт ба UI дүрслэл 14
6.1 Юуг өөрчилж туршиж болох вэ..... 11
6.2 Юуг ажиглах вэ……..... 11
6.3 Worker-ийн ажиллагаа..... 11
7. Туршилтын арга 15
8. Үр дүн 16
9. Хэлэлцүүлэг 17
10. Дүгнэлт 18
Ашигласан материал 18
Хавсралт А 19

Хураангуй

Асуудал: Энэхүү төслийн зорилго нь F.CSM302 Үйлдлийн систем хичээлийн Threads & Concurrency сэдвийг практикт харуулах интерактив эрэмбэлэлтийн програм боловсруулах юм.

Шийдэл: Програм нь хэрэглэгчийн оруулсан бүхэл тоон жагсаалтыг хоёр хэсэгт хувааж, тус бүрийг тусдаа sorting worker-ээр эрэмбэлээд, дараа нь merge worker ашиглан нэг жагсаалт болгон нэгтгэнэ.

Туршилт: Хэрэгжүүлэлтэд HTML, CSS, JavaScript болон Web Worker ашигласан. Үндсэн thread нь хэрэглэгчийн интерфейс, оролтын шалгалт, worker-үүдийн мессеж солилцоо, ажиллагааны төлөвийг зохицуулна. Sorting worker-үүд Insertion Sort алгоритм ашиглах бөгөөд merge worker нь хоёр эрэмбэлэгдсэн хэсгийг нэгтгэнэ.

Интерфейс нь гараар өгөгдөл оруулах, санамсаргүй өгөгдөл үүсгэх, Start, Pause, Resume, Reset үйлдэл хийх, хөдөлгөөнт дүрслэлийн хурдыг өөрчлөх боломжтой. Тестийн төлөвлөгөөнд энгийн, урвуу эрэмбэлэгдсэн, давхардсан утгатай, сөрөг тоотой болон буруу оролтын тохиолдлууд багтсан.

Үр дүн: Төслийн ажиллагааг шалгахын тулд өгөгдөл эрэмбэлэх үндсэн функц, алдаатай оролт боловсруулах боломж болон хэрэглэгчийн интерфейсийн удирдлагуудыг туршив. Туршилтын үр дүнг хүлээгдэж буй үр дүнтэй харьцуулан үнэлэв.

1. Удиртгал

1.1 Зорилго

Төслийн зорилго нь олон урсгалт боловсруулалтын үндсэн ойлголтыг Web Worker ашиглан дүрслэн харуулж, жагсаалтын хэсгүүдийг зэрэг эрэмбэлэн нэгтгэх програм хэрэгжүүлэх юм.

1.2 Ач холбогдол

Энэхүү ажил нь үндсэн thread болон worker-ийн ялгаа, мессежээр өгөгдөл дамжуулах арга, ажлыг хэсэгчилж гүйцэтгэх болон үр дүнг нэгтгэх ажиллагааг ойлгоход тусална. Хөдөлгөөнт дүрслэл нь алгоритмын үе шатыг ажиглах боломж олгоно.

1.3 Хамрах хүрээ

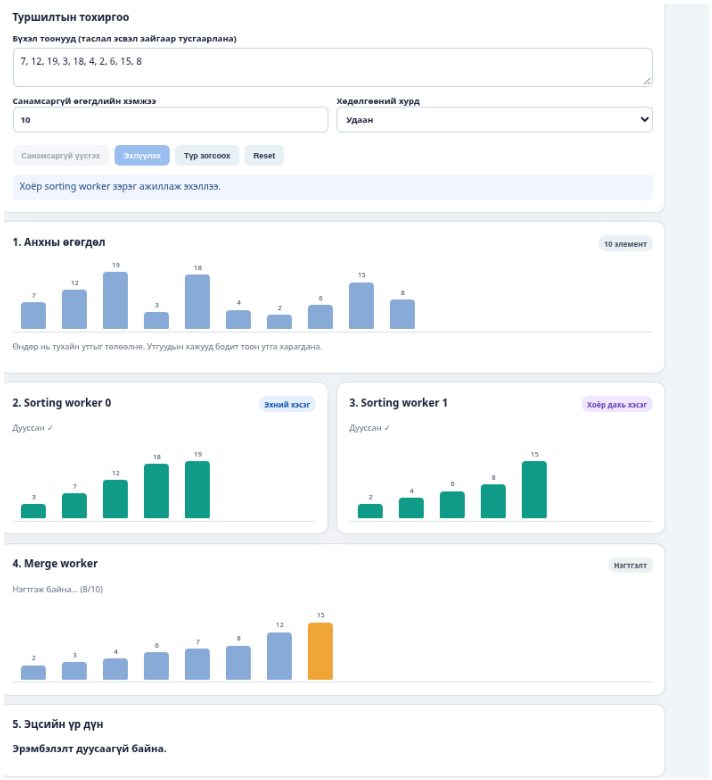
- Бүхэл тоон жагсаалт хүлээн авах болон оролтыг шалгах.
- Жагсаалтыг хоёр хэсэгт хувааж, хоёр sorting worker-ээр эрэмбэлэх.
- Merge worker ашиглан хэсгүүдийг нэгтгэх.
- Хөдөлгөөнт явцыг харуулах, удирдлага болон хурдны тохиргоо өгөх.
- Эцсийн үр дүнгийн эрэмбэ болон элементийн хадгалалтыг шалгах.

2. Сурах бичгийн шаардлага

Төсөл нь үйлдлийн системийн сурах бичгийн Threads & Concurrency бүлэгт авч үздэг олон урсгалт ажиллагаа, тусдаа гүйцэтгэлийн урсгалуудын хоорондын зохицуулалт, өгөгдөл дамжуулалт болон үр дүн нэгтгэх ойлголтуудтай холбоотой.

Энэхүү хэрэгжүүлэлт нь браузерын Web Worker API ашигладаг. Web Worker нь ердийн үйлдлийн системийн thread-ийн бүх шинжийг шууд илэрхийлэхгүй боловч үндсэн интерфейсийн гүйцэтгэлээс ажлыг тусгаарлах, мессежээр харилцах загварыг харуулна.

2.1 Шаардлагуудын хэрэгжилт



Шаардлага	Төслийн шийдэл
Ажлыг хуваах	Оролтын массивыг хоёр хэсэг болгоно.
Зэрэг гүйцэтгэх	Хоёр sorting worker хэсгүүдийг тус тус боловсруулна.
Үр дүн нэгтгэх	Merge worker эрэмбэлэгдсэн хэсгүүдийг нэгтгэнэ.
Мессеж солилцоо	postMessage() болон message event ашиглана.
UI удирдлага	Start, Pause, Resume, Reset болон хурдны тохиргоо.
Оролтын шалгалт	Хоосон, буруу форматтай, хэт урт, аюулгүй бус бүхэл тоог шалгана.

3. Онолын үндэс

3.1. Үйлдлийн системийн ойлголт

Үйлдлийн систем (Operating System) нь компьютерийн техник хангамж болон хэрэглээний програмуудын хооронд зуучлан ажиллаж, компьютерийн нөөцийг удирдах үндсэн системийн програм хангамж юм. Үйлдлийн систем нь процессор, санах ой, файл, оролт гаралтын төхөөрөмж болон програмуудын ажиллагааг зохицуулдаг.

Үйлдлийн системийн үндсэн үүргүүдэд процессын удирдлага, санах ойн хуваарилалт, файлын системийн удирдлага, төхөөрөмжийн зохицуулалт болон програмуудын хоорондын харилцааг хангах зэрэг багтана.

Орчин үеийн үйлдлийн системүүд нь олон програмыг зэрэг ажиллуулах боломжтой. Үүний нэг хэлбэр нь олон урсгалт боловсруулалт (Multithreading) юм. Урсгал (Thread) гэдэг нь процессын хүрээнд гүйцэтгэгдэж буй үйл ажиллагааны нэг дараалал бөгөөд нэг процесс олон урсгалтай байж болно. Урсгалууд нь тухайн процессын санах ойн орчныг хуваалцдаг боловч тус бүр өөрийн гүйцэтгэлийн дараалалтай байна.

Энэхүү төслийн хүрээнд олон урсгалт боловсруулалтын зарчмыг эрэмбэлэлтийн програмд хэрэгжүүлсэн. Өгөгдлийг хоёр хэсэгт хувааж, тус бүрийг Web Worker ашиглан зэрэг эрэмбэлсний дараа үр дүнг нэгтгэх замаар зэрэгцээ боловсруулалтын үндсэн санааг хэрэгжүүлсэн.

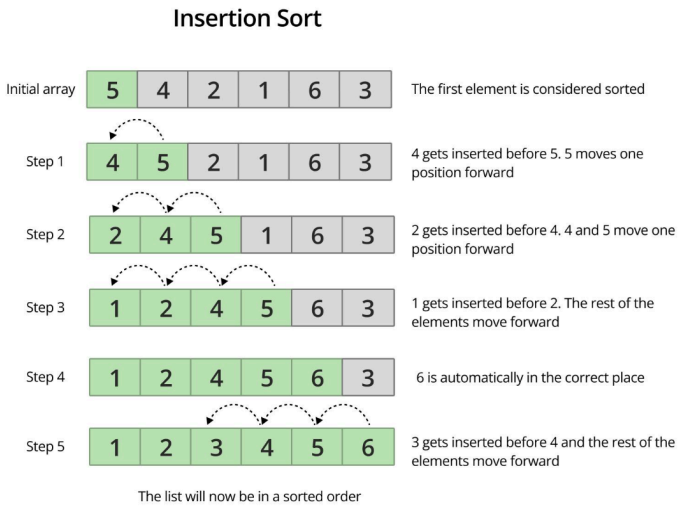
3.2. Алгоритм

Алгоритм гэдэг нь тодорхой асуудлыг шийдвэрлэхийн тулд дараалсан, төгсгөлөг алхмуудыг гүйцэтгэх заавар юм. Алгоритм нь оролтын өгөгдлийг боловсруулж, шаардлагатай гаралтын үр дүнг гаргана.

Алгоритмыг боловсруулахдаа зөв байдал, ойлгомжтой байдал, үр ашигтай ажиллагаа болон төгсгөлөг хугацаанд дуусах шинжийг харгалзан үздэг. Алгоритмын үр ашгийг үнэлэхдээ хугацааны нийлмэл байдал (Time Complexity) болон санах ойн нийлмэл байдал (Space Complexity)-ыг ашигладаг. Эдгээрийг ихэвчлэн Big O тэмдэглэгээгээр илэрхийлнэ.

Энэхүү төсөлд массивын элементүүдийг өсөх дарааллаар эрэмбэлэх, хоёр хэсгийн эрэмбэлэгдсэн үр дүнг нэгтгэх алгоритмуудыг ашигласан.

Оруулах эрэмбэлэлт (Insertion Sort)



Insertion Sort нь массивын элементүүдийг нэг нэгээр нь авч, өмнө нь эрэмбэлэгдсэн хэсэгт тохирох байрлалд оруулах зарчимтай алгоритм юм. Энэ алгоритм нь хэрэгжүүлэхэд хялбар бөгөөд бага хэмжээний өгөгдөлд тохиромжтой.

- Хамгийн сайн тохиолдлын хугацааны нийлмэл байдал: O(n)
- Дундаж тохиолдлын хугацааны нийлмэл байдал: $O(n^2)$
- Хамгийн муу тохиолдлын хугацааны нийлмэл байдал: $O(n^2)$
- Нэмэлт санах ойн нийлмэл байдал: O(1)

Энд n нь эрэмбэлэх элементийн тоо юм.

Нэгтгэх алгоритм (Merge)

Хоёр тусдаа эрэмбэлэгдсэн жагсаалтыг нэг эрэмбэлэгдсэн жагсаалт болгохын тулд Merge алгоритмыг ашиглана. Хоёр жагсаалтын эхний элементүүдийг харьцуулж, бага утгыг үр дүнгийн жагсаалтад нэмнэ. Нэг жагсаалт дуусах хүртэл энэ үйлдлийг давтаж, үлдсэн элементүүдийг үр дүнд нэмж дуусгана.

Хэрэв хоёр жагсаалтын нийт элементийн тоо n бол нэгтгэх ажиллагааны хугацааны нийлмэл байдал O(n) байна.

Төслийн алгоритмын ажиллагаа

1. Хэрэглэгч бүхэл тоон өгөгдлийг оруулна.
2. Оролтын өгөгдлийн зөв эсэх болон элементийн тоог шалгана.
3. Өгөгдлийг хоёр хэсэгт хуваана.
4. Хоёр хэсгийг тус тусад нь Insertion Sort ашиглан эрэмбэлнэ.
5. Эрэмбэлэгдсэн хоёр хэсгийг Merge алгоритмаар нэгтгэнэ.
6. Эцсийн үр дүнг хэрэглэгчийн интерфейс дээр харуулна.

Энэ арга нь өгөгдлийг хэсэгчлэн боловсруулах, Web Worker-уудын тусламжтай үндсэн интерфейсийн ажиллагааг тусгаарлах боломж олгодог.

3.3. Өгөгдлийн бүтэц

Өгөгдлийн бүтэц гэдэг нь өгөгдлийг компьютерт хадгалах, зохион байгуулах болон боловсруулахад ашигладаг хэлбэр юм. Зөв өгөгдлийн бүтцийг сонгох нь програмын ажиллагааны хурд, санах ойн хэрэглээ болон алгоритмын үр ашигт нөлөөлдөг. Энэхүү төсөлд дараах өгөгдлийн бүтцүүдийг ашигласан.

Массив (Array): Массив нь ижил төрлийн эсвэл нэг төрлийн утгуудыг дараалалтай хадгалах өгөгдлийн бүтэц юм. JavaScript хэлэнд массивыг Array төрлөөр илэрхийлдэг. Төслийн оролтын тоонууд, эрэмбэлэгдсэн хэсгүүд болон эцсийн үр дүнг массив хэлбэрээр хадгална.

Жагсаалт ба дэд массив: Өгөгдлийг хоёр хэсэгт хуваах үед эх массивын элементүүдийг дэд массивуудад байрлуулна. Хэсэг бүрийг тусдаа боловсруулж, эрэмбэлсний дараа нэгтгэнэ. Энэ нь өгөгдлийг зэрэгцээ боловсруулахад тохиромжтой зохион байгуулалт юм.

Мессежээр дамжуулах өгөгдөл: Web Worker нь үндсэн JavaScript урсгалаас тусдаа гүйцэтгэлийн орчинд ажилладаг. Үндсэн урсгал болон Worker хооронд өгөгдөл дамжуулахдаа postMessage() болон onmessage зэрэг механизмыг ашиглаж болно. Төслийн хувьд оролтын өгөгдөл, эрэмбэлэгдсэн үр дүн болон ажиллагааны төлөвийг эдгээр мессежээр дамжуулдаг.

Өгөгдлийн бүтцийн сонголтын үндэслэл: Массив нь эрэмбэлэх, элементүүдийг харьцуулах, өгөгдлийг хэсэглэх болон үр дүнг нэгтгэхэд тохиромжтой. Харин Web Worker ашигласнаар үндсэн интерфейсийн урсгал болон өгөгдөл боловсруулах ажиллагааг тусгаарлах боломжтой. Иймээс массив болон мессежээр өгөгдөл дамжуулах аргыг сонгосон нь төслийн зорилгод нийцэж байна.

4. Системийн зохиомж

Энэхүү төсөл нь олон урсгалт эрэмбэлэлтийн ажиллагааг харуулах зориулалттай бөгөөд хэрэглэгчийн интерфейс, өгөгдөл боловсруулах логик болон Web Worker-ууд гэсэн үндсэн хэсгүүдээс бүрдэнэ. Компонентууд хоорондоо харилцан ажиллаж, хэрэглэгчийн оруулсан өгөгдлийг хүлээн авах, эрэмбэлэх, нэгтгэх болон үр дүнг дэлгэцэд харуулах үйл явцыг хэрэгжүүлдэг.

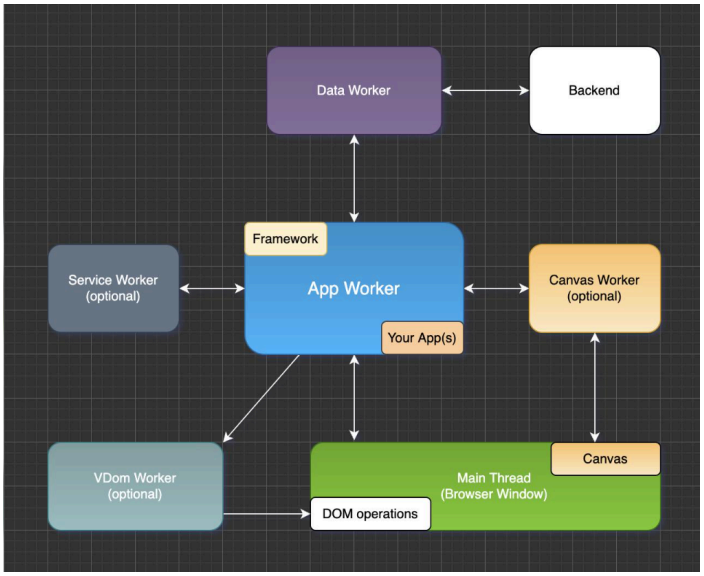
Хэрэглэгчийн интерфейс (User Interface) нь өгөгдөл оруулах талбар, удирдлагын товчлуурууд болон эрэмбэлэлтийн үр дүнг харуулах хэсгүүдээс бүрдэнэ. Хэрэглэгч Start товчийг дарснаар эрэмбэлэлтийн ажиллагаа эхэлнэ. Pause, Resume, Reset болон Speed удирдлагууд нь ажиллагааг түр зогсоох, үргэлжлүүлэх, анхны төлөвт буцаах болон хөдөлгөөнт дүрсний хурдыг тохируулах боломж олгодог.

Өгөгдөл боловсруулах хэсэг нь хэрэглэгчийн оруулсан утгуудыг шалгаж, боловсруулахад бэлтгэнэ. Өгөгдөл хүчинтэй бол массивыг хоёр хэсэгт хувааж, тус бүрийг эрэмбэлэхээр Web Worker-ууд руу дамжуулна. Ингэснээр эрэмбэлэлтийн боловсруулалт хэрэглэгчийн интерфейсийн үндсэн JavaScript урсгалаас тусгаарлагдана.

Эрэмбэлэлтийн Worker-ууд нь өгөгдлийн хоёр хэсгийг тус тусад нь Insertion Sort алгоритмаар эрэмбэлнэ. Хоёр Worker-ийн үр дүн бэлэн болсны дараа нэгтгэх Worker эсвэл нэгтгэх логик нь хоёр эрэмбэлэгдсэн хэсгийг Merge алгоритмаар нэг массив болгоно. Үр дүнд нь бүх элемент өсөх дарааллаар байрлана.

Өгөгдлийн урсгал нь дараах дарааллаар явагдана:

1. Хэрэглэгч интерфейсээр дамжуулан бүхэл тоон өгөгдөл оруулна.
2. Програм оролтын өгөгдлийн зөв эсэхийг шалгана.
3. Өгөгдлийг хоёр хэсэгт хувааж, эрэмбэлэх Worker-ууд руу мессежээр дамжуулна.
4. Worker-ууд тус тусын хэсгийг эрэмбэлж, үр дүнг үндсэн програмд буцаана.
5. Хоёр үр дүнг нэгтгэж, эцсийн эрэмбэлэгдсэн массивыг гаргана.
6. Хэрэглэгчийн интерфейс дээр үр дүн болон эрэмбэлэлтийн явцыг харуулна.



Үндсэн JavaScript урсгал болон Worker-уудын хооронд өгөгдөл дамжуулахад postMessage() болон onmessage зэрэг механизмыг ашигладаг. Энэ нь компонентуудын хооронд мэдээлэл солилцох боломжийг бүрдүүлнэ. Мөн хэрэглэгчийн удирдлагын үйлдлүүд нь програмын төлөв болон хөдөлгөөнт дүрсний ажиллагаанд нөлөөлнө.

Дүгнэлт: Компонентуудын үүргийг тусгаарлан зохион байгуулснаар өгөгдөл хүлээн авах, эрэмбэлэх, нэгтгэх болон үр дүнг харуулах үйл явцыг ойлгомжтой болгож, олон урсгалт боловсруулалтын ажиллагааг дүрслэн харуулах боломж бүрдсэн.

5. Хэрэгжилт

5.1 Хэл, сан, орчин

- HTML — интерфейсийн бүтэц.
- CSS — интерфейсийн загвар, хөдөлгөөнт дүрслэл.
- JavaScript — өгөгдөл боловсруулах болон удирдлагын логик.
- Web Worker — эрэмбэлэлт болон нэгтгэлтийн ажиллагааг тусгаарлах.
- postMessage() — үндсэн thread болон worker хооронд өгөгдөл дамжуулах.

5.2 Массив хуваах

Оролтын массивыг хоёр хэсэг болгон хуваасны дараа хэсэг бүрийг өөр worker-д дамжуулна. Хэсгүүдийн хэмжээ тэнцүү байх албагүй; нийт элементийн тоо сондгой үед нэг хэсэгт нөгөө хэсгээс нэг элементээр илүү байж болно.

5.3 Worker-ийн ажиллагаа

Sorting worker нь өөрт ирсэн массивыг Insertion Sort ашиглан эрэмбэлж, үр дүнг мессежээр буцаана. Хоёр үр дүн бэлэн болсон үед үндсэн програм merge worker руу өгөгдөл илгээнэ.

5.4 Merge worker

Merge worker нь хоёр эрэмбэлэгдсэн массивыг нэг массив болгон нэгтгэнэ. Алгоритм нь хоёр хэсгийн эхний ашиглагдаагүй элементүүдийг харьцуулж, бага элементийг дараагийн байрлалд оруулна. Нэг хэсэг дуусмагц нөгөө хэсгийн үлдсэн элементүүдийг үр дүнд нэмнэ.

5.5 Удирдлага ба хамгаалалт

Start, Pause, Resume болон Reset нь ажиллагааны төлөвийг удирдана. Хурдны тохиргоо нь хөдөлгөөнт дүрслэлийн хугацаанд нөлөөлнө. runId нь өмнөх ажиллагаанаас ирсэн хоцорсон мессежийг шинэ ажиллагааны үр дүн гэж хүлээн авахаас хамгаална.

5.6 Репозиторийн бүтэц

Файл/хавтас	**Зориулалт**
src/index.html	Хэрэглэгчийн интерфейсийн HTML
src/styles.css	Загвар болон дүрслэл
src/app.js	Үндсэн програмын логик
src/workers-code.js	Worker-ийн код
tests/test-cases.md	Тестийн тохиолдлын жагсаалт
README.md	Төслийн тайлбар болон ажиллуулах заавар

6. Туршилт ба UI дүрслэл

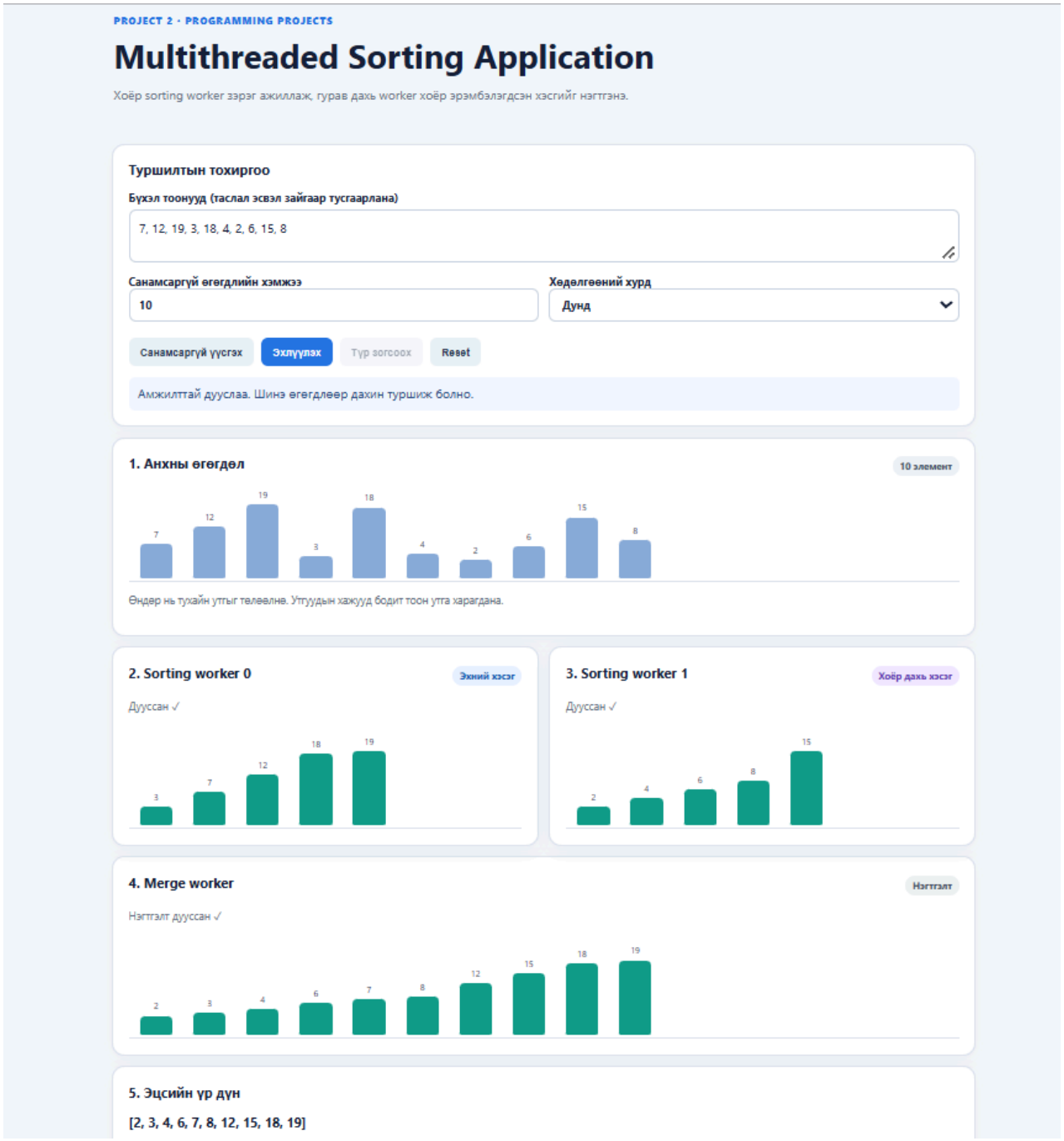
6.1 Юуг өөрчилж туршиж болох вэ

- Оролтын массивын хэмжээ болон дараалал.
- Сөрөг, эерэг, давхардсан утгууд.
- Хөдөлгөөнт дүрслэлийн хурд.
- Start, Pause, Resume болон Reset үйлдлүүд.
- Хоосон болон буруу форматтай оролт.

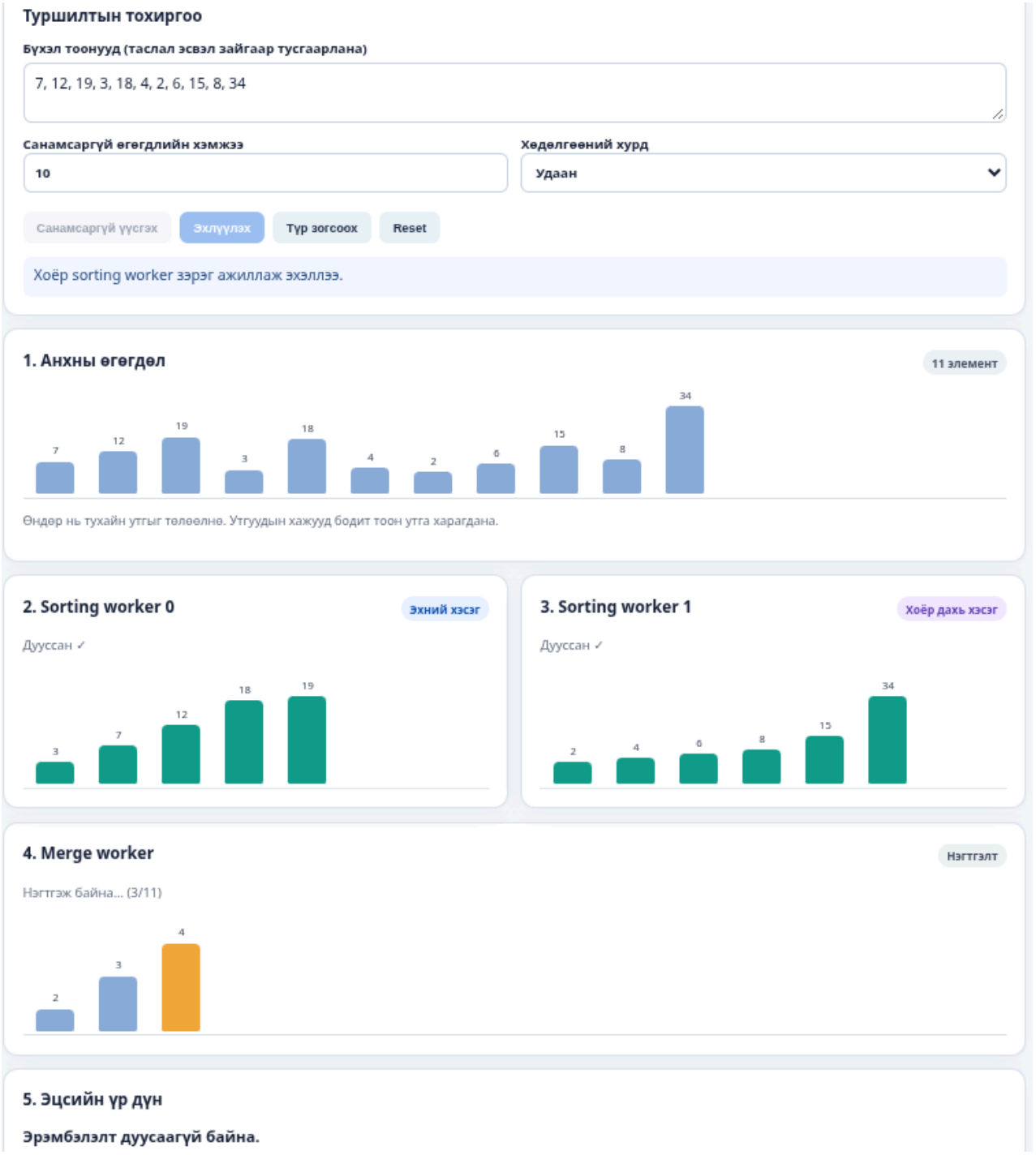
6.2 Юуг ажиглах вэ

- Хоёр sorting worker тус бүр өөрийн хэсгийг зөв эрэмбэлж байгаа эсэх.
- Merge worker-ийн үр дүн бүхэлдээ өсөх дараалалтай эсэх.
- Эхний массивын элемент бүр үр дүнд хадгалагдсан эсэх.
- Pause/Resume/Reset үед UI болон worker-ийн төлөв зөв өөрчлөгдөж байгаа эсэх.
- Буруу оролтын үед алдааны мэдэгдэл гарч байгаа эсэх.

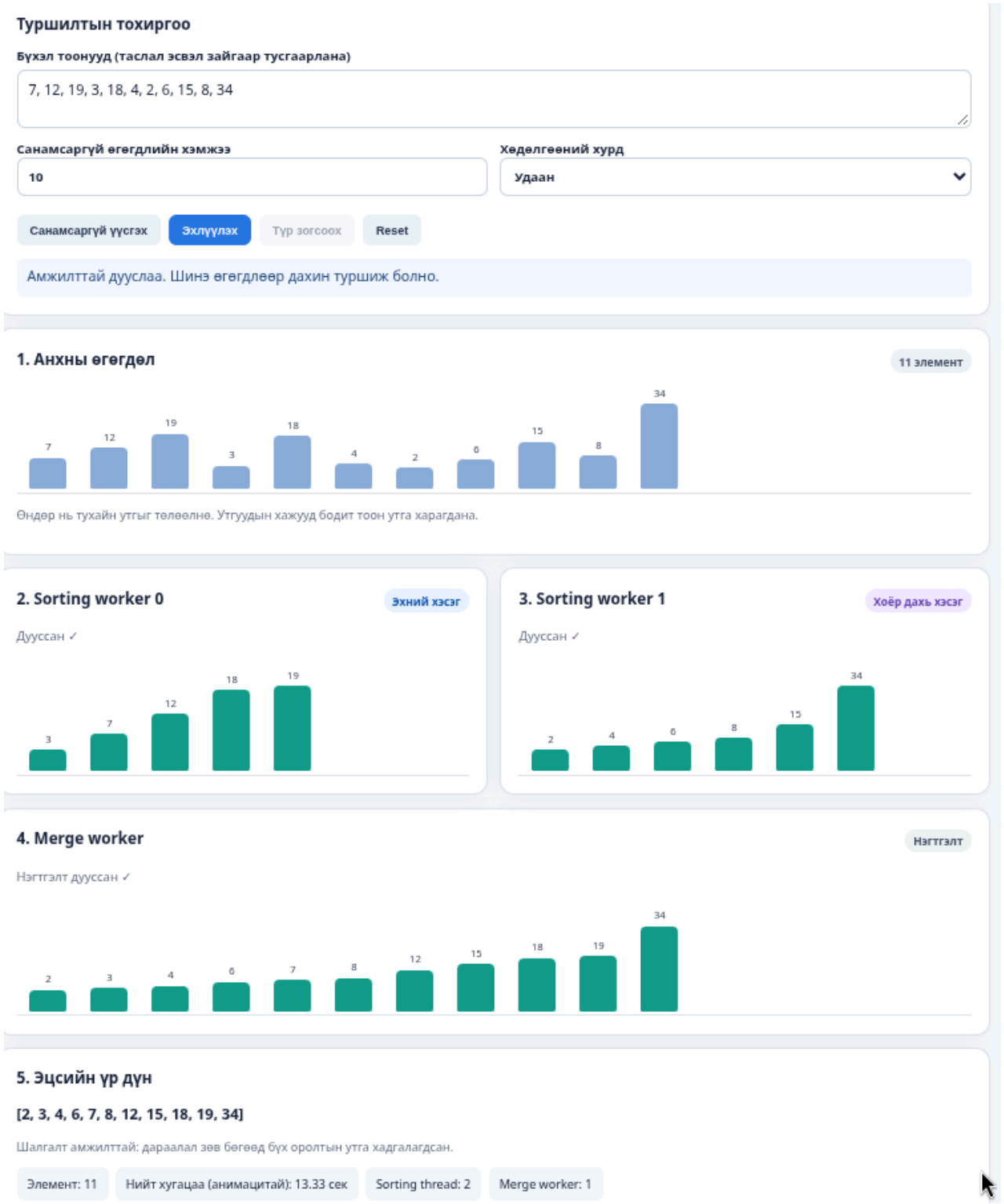
6.3 UI-ийн дэлгэцийн зураг



Эрэмбэлэлт явагдаж буй үеийн дэлгэцийн зураг



Эцсийн эрэмбэлэгдсэн үр дүнгийн дэлгэцийн зураг



7. Туршилтын арга

7.1. Туршилтын орчин

Энэхүү төслийг веб орчинд ажиллах байдлаар боловсруулсан. Програмын хэрэглэгчийн интерфейсийг HTML, CSS болон JavaScript ашиглан хэрэгжүүлж, өгөгдлийг зэрэгцээ боловсруулахад Web Worker технологийг ашигласан. Програмыг локал веб серверээр ажиллуулж, веб хөтөч дээр туршилт хийсэн.

Туршилтын орчинд дараах технологи, хэрэгслүүдийг ашиглав.

- HTML: хэрэглэгчийн интерфейсийн бүтцийг тодорхойлох.
- CSS: интерфейсийн загвар болон хөдөлгөөнт дүрсийг дүрслэх.
- JavaScript: өгөгдөл боловсруулах, эрэмбэлэх болон хэрэглэгчийн үйлдлийг удирдах.
- Web Worker: өгөгдлийн хэсгүүдийг тусдаа гүйцэтгэлийн орчинд боловсруулах.
- Локал веб сервер: програмын файлуудыг веб хөтөчөөр дамжуулан ажиллуулах.

Туршилтын явцад эрэмбэлэлтийн зөв байдал, алдаатай оролтын боловсруулалт, хэрэглэгчийн интерфейсийн удирдлага болон хөдөлгөөнт дүрсний ажиллагааг шалгасан.

7.2. Туршилтын өгөгдөл

Програмын ажиллагааг үнэлэхийн тулд өөр өөр шинж чанартай оролтын өгөгдлүүдийг ашигласан. Үүнд эрэмбэлэгдсэн дараалалтай, урвуу дараалалтай, давхардсан утгатай, сөрөг тоо агуулсан болон цөөн элементтэй өгөгдлүүд багтана. Мөн програмын алдаа боловсруулах чадварыг шалгахын тулд хүчингүй болон хоосон оролт, элементийн тооны хязгаараас хэтэрсэн өгөгдлийг авч үзсэн.

Хүснэгт 7.1. Туршилтын өгөгдлийн жишээ

№	Туршилтын төрөл	Оролтын өгөгдөл	Хүлээгдэж буй үр дүн
1	Энгийн өгөгдөл	5, 2, 8, 1, 3	1, 2, 3, 5, 8
2	Эрэмбэлэгдсэн өгөгдөл	1, 2, 3, 4, 5	1, 2, 3, 4, 5
3	Урвуу дараалал	9, 7, 5, 3, 1	1, 3, 5, 7, 9
4	Давхардсан утга	4, 2, 4, 1, 2	1, 2, 2, 4, 4
5	Сөрөг тоо	−5, 3, −1, 0	−5, −1, 0, 3
6	Ижил утгууд	7, 7, 7, 7, 7	7, 7, 7, 7, 7
7	Хоёр элемент	5, 5	5, 5

8	Хүчингүй оролт	1, abc, 3	Алдааны мэдэгдэл
9	Хоосон оролт	Өгөгдөлгүй	Хоосон оролтын мэдэгдэл
10	Хязгаараас олон элемент	200-аас олон элемент	Хязгаарын мэдэгдэл

Эдгээр өгөгдөл нь эрэмбэлэлтийн алгоритмын зөв ажиллагааг болон програмын оролтын шалгалтыг үнэлэхэд ашиглагдана. Хүчингүй оролтын үед програм нь зохих мэдэгдэл харуулах ёстой бөгөөд зөв оролтын үед бүх элементийг өсөх дарааллаар эрэмбэлэх шаардлагатай.

7.3. Хүлээгдэж буй үр дүн

Туршилтын үр дүнд програм нь өгөгдлийг зөв эрэмбэлж, хэрэглэгчийн интерфейсээр дамжуулан үр дүнг ойлгомжтой харуулах ёстой. Өгөгдлийг хоёр хэсэгт хуваан боловсруулах болон эрэмбэлэгдсэн хэсгүүдийг нэгтгэх ажиллагаа нь эцсийн үр дүнгийн зөв байдалд нөлөөлөхгүй байх шаардлагатай.

Хүлээгдэж буй үндсэн үр дүнг дараах байдлаар тодорхойлов.

1. Эрэмбэлэлтийн зөв байдал: хүчинтэй оролтын бүх элемент өсөх дарааллаар зөв байрлах.
2. Өгөгдлийн бүрэн бүтэн байдал: эрэмбэлэлтийн дараа элемент алдагдахгүй, давхардсан утгууд хадгалагдах.
3. Алдаа боловсруулах: хоосон болон хүчингүй оролтын үед хэрэглэгчид ойлгомжтой мэдэгдэл харуулах.
4. Зэрэгцээ боловсруулалт: өгөгдлийн хоёр хэсгийг тус тусад нь Worker ашиглан боловсруулах.
5. Үр дүнг нэгтгэх: эрэмбэлэгдсэн хоёр хэсгийг нэг зөв дараалалтай массив болгох.
6. Интерфейсийн ажиллагаа: Start, Pause, Resume, Reset болон Speed удирдлагууд зориулалтын дагуу ажиллах.
7. Хэрэглэгчийн ойлгомжтой байдал: эрэмбэлэлтийн явц болон эцсийн үр дүнг интерфейс дээр харуулах.

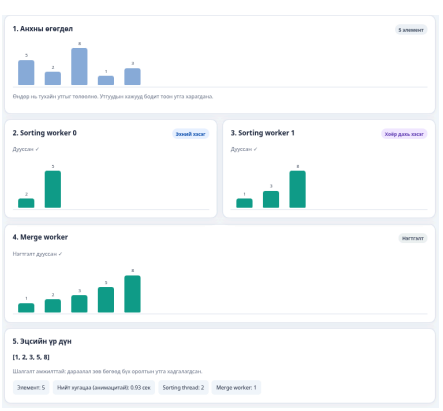
Эдгээр шалгуурыг бодит туршилтын үр дүнтэй харьцуулж, програмын ажиллагааг үнэлнэ. Хүлээгдэж буй үр дүн нь туршилтаар баталгаажсан бодит үр дүнгээс ялгаатай байж болох тул эцсийн үнэлгээг зөвхөн хийж гүйцэтгэсэн туршилтын мэдээлэлд үндэслэн гаргана.

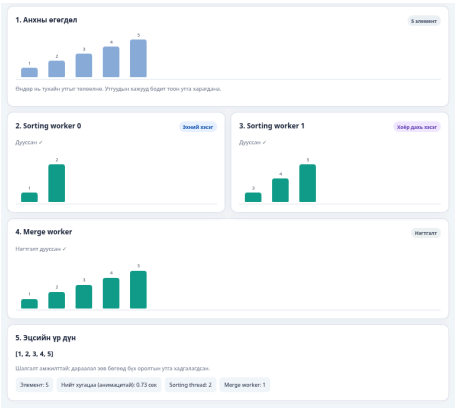
8. ҮР ДҮН

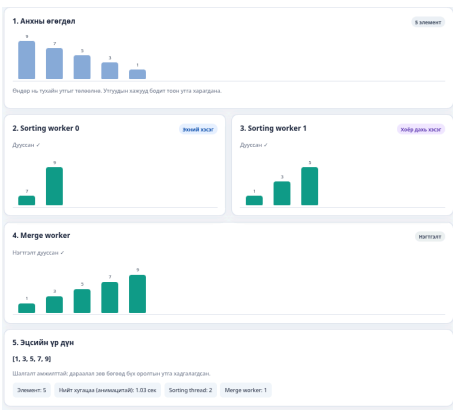
Төслийн ажиллагааг шалгахын тулд өгөгдөл эрэмбэлэх үндсэн функц, алдаатай оролт боловсруулах боломж болон хэрэглэгчийн интерфейсийн удирдлагуудыг туршив. Туршилтын үр дүнг хүлээгдэж буй үр дүнтэй харьцуулан үнэлэв.

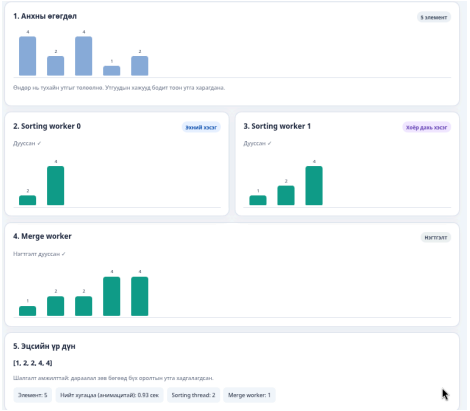
Хүснэгт 8.1. Эрэмбэлэлтийн функциональ тестийн үр дүнХүснэгт

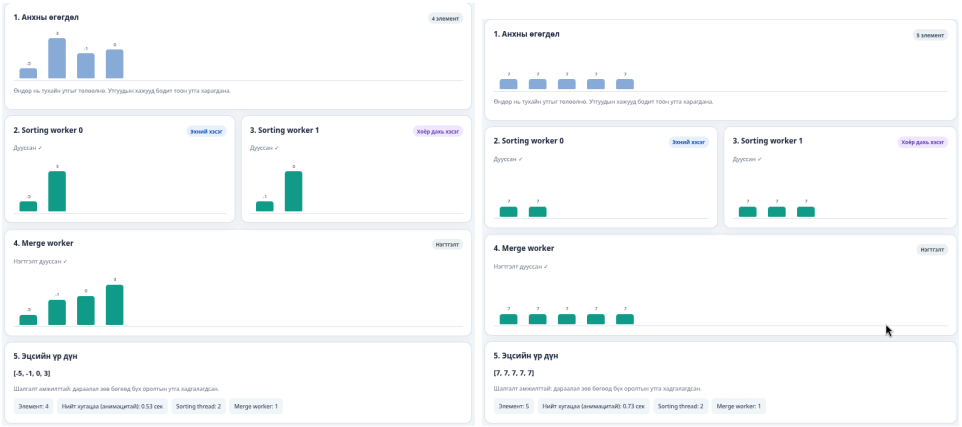
№	Туршилтын төрөл	Оролтын өгөгдөл	Хүлээгдэж буй үр дүн	Бодит үр дүн	Төлөв
T1	Энгийн эрэмбэлэлт	5, 2, 8, 1, 3	1, 2, 3, 5, 8	1, 2, 3, 5, 8	Амжилттай
T2	Эрэмбэлэгдсэн оролт	1, 2, 3, 4, 5	1, 2, 3, 4, 5	1, 2, 3, 4, 5	Амжилттай
T3	Урвуу дараалал	9, 7, 5, 3, 1	1, 3, 5, 7, 9	1, 3, 5, 7, 9	Амжилттай
T4	Давхардсан утга	4, 2, 4, 1, 2	1, 2, 2, 4, 4	1, 2, 2, 4, 4	Амжилттай
T5	Сөрөг утга	−5, 3, −1, 0	−5, −1, 0, 3	−5, −1, 0, 3	Амжилттай
T6	Ижил утгууд	7, 7, 7, 7, 7	7, 7, 7, 7, 7	7, 7, 7, 7, 7	Амжилттай
T7	Сондгой тооны элемент	1, 3, 2001	1, 3, 2001	1, 3, 2001	Амжилттай
T8	Хоёр элементтэй оролт	5, 5	5, 5	5, 5	Амжилттай

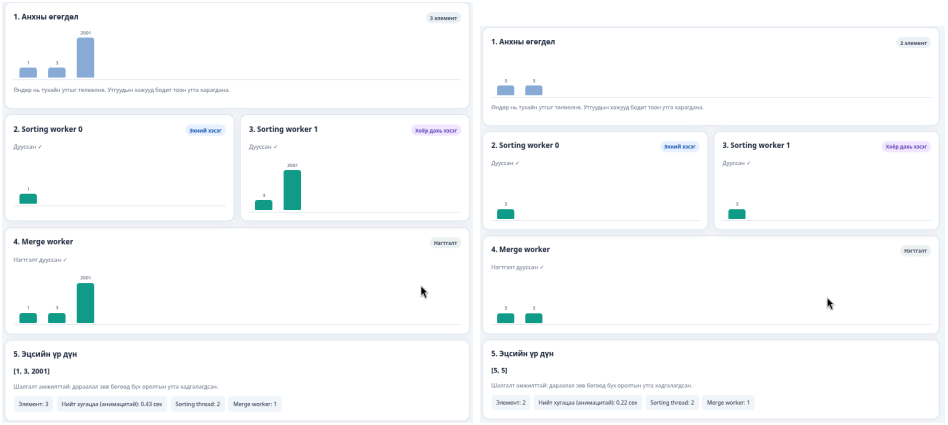






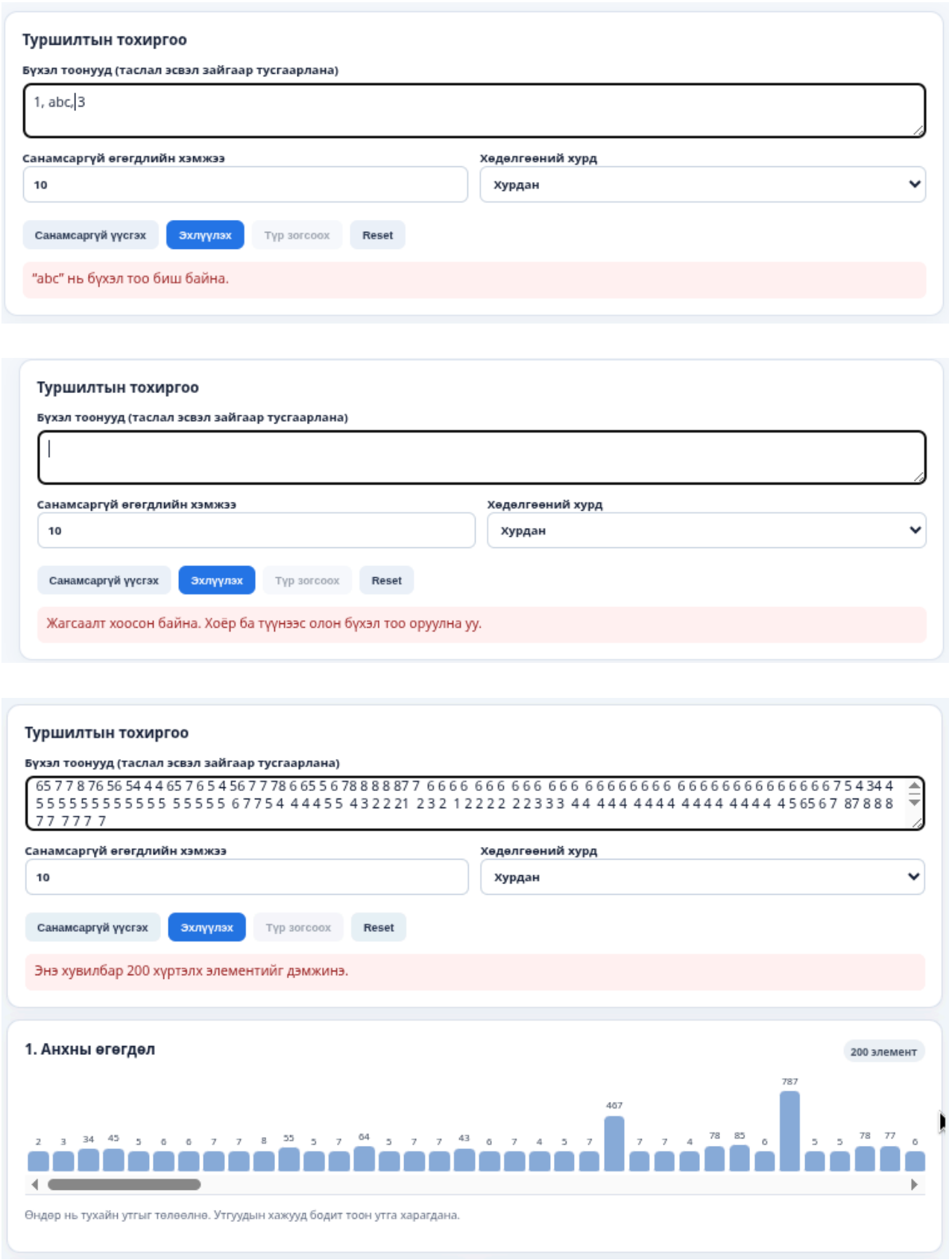






8.2. Алдаатай болон хязгаарын оролтын тестийн үр дүн

№	Туршилтын төрөл	Оролт	Хүлээгдэж буй үр дүн	Бодит үр дүн	Төлөв
E1	Бүхэл тоон бус утга	1, abc, 3	Алдааны мэдэгдэл харуулах	“abc” нь бүхэл тоо биш гэсэн мэдэгдэл харуулсан	Амжилттай
E2	Хоосон оролт	Хоосон жагсаалт	Ойлгомжтой мэдэгдэл харуулах	“Жагсаалт хоосон байна. Хоёр ба түүнээс олон бүхэл тоо оруулна уу.” гэсэн мэдэгдэл харуулсан	Амжилттай
E3	Элементийн хязгаар хэтрэх	200-аас олон элемент	Хязгаарын мэдэгдэл харуулах	“Энэ хувилбар 200 хүртэлх элементийг дэмжинэ” гэсэн мэдэгдэл харуулсан	Амжилттай



Хүснэгт 8.3. Хэрэглэгчийн интерфейсийн (UI) тестийн үр дүн

№	Туршилтын төрөл	Хүлээгдэж буй үр дүн	Бодит үр дүн	Төлөв
U1	Start	Эрэмбэлэлтийг эхлүүлэх	Эрэмбэлэлтийн ажиллагаа амжилттай эхэлсэн	Амжилттай
U2	Pause	Ажиллагааг түр зогсоох	Ажиллагаа амжилттай түр зогссон	Амжилттай
U3	Resume	Түр зогсоосон ажиллагааг үргэлжлүүлэх	Ажиллагаа амжилттай үргэлжилсэн	Амжилттай
U4	Reset	Програмыг эхний төлөвт буцаах	Эхний төлөвт амжилттай буцсан	Амжилттай
U5	Speed	Хөдөлгөөнт дүрсний хурдыг өөрчлөх	Хурдны тохиргоо амжилттай ажилласан	Амжилттай

8.1. Үр дүнгийн тайлбар

Туршилтын үр дүнгээс харахад програм нь өсөх дарааллаар эрэмбэлэх үндсэн үйлдлийг зөв гүйцэтгэж, эрэмбэлэгдсэн, урвуу дарааллатай, сөрөг тоо агуулсан болон давхардсан элементтэй өгөгдлийг зөв боловсруулсан. Мөн хоёр элементтэй болон сондгой тооны элементтэй оролтын үед зөв үр дүн гаргасан.

Алдаатай өгөгдөл, хоосон оролт болон 200 элементийн хязгаараас хэтэрсэн оролтыг боловсруулах үед хэрэглэгчид ойлгомжтой мэдэгдэл харуулсан. Хэрэглэгчийн интерфейсийн Start, Pause, Resume, Reset болон Speed удирдлагуудын ажиллагааг шалгаж, бүгдийг амжилттай гэж үнэлэв.

Ерөнхийдөө хийсэн туршилтууд нь програмын эрэмбэлэлтийн үндсэн ажиллагаа, оролтын шалгалт болон хэрэглэгчийн интерфейсийн удирдлагууд хэвийн ажиллаж байгааг харууллаа.

9. Хэлэлцүүлэг

9.1 Үр дүнгийн тайлбар

Туршилтын үр дүнг тайлбарлахдаа зөвхөн эцсийн массив өсөх дарааллаар эрэмбэлэгдсэн эсэхийг бус, оролтын элементүүд алдагдаагүй болон давхардаагүй хадгалагдсан эсэхийг мөн авч үзнэ. UI удирдлагуудын ажиллагааг тусад нь гараар шалгана.

9.2 Хязгаарлалт

- Insertion Sort нь их хэмжээний өгөгдөлд $O(n^2)$ хугацаа шаардаж болно.
- Web Worker нь браузерын орчинд ажилладаг бөгөөд OS-ийн native thread-тэй яг ижил биш.
- Хөдөлгөөнт дүрслэл нь алгоритмын бодит гүйцэтгэлийн хугацааг шууд хэмжих хэрэгсэл биш.
- Туршилтын бодит үр дүн, хөтчийн орчин болон дэлгэцийн нотолгоог тайланд нөхөж оруулах шаардлагатай.

10. Дүгнэлт

Энэхүү төслөөр Threads & Concurrency сэдвийн ойлголтыг JavaScript Web Worker ашиглан практикт харуулах зорилго тавьсан. Оролтын жагсаалтыг хоёр хэсэгт хувааж, хоёр sorting worker-ээр эрэмбэлэн, merge worker-ээр нэгтгэх архитектурыг хэрэгжүүлсэн.

Програмын интерфейс нь оролт оруулах, санамсаргүй өгөгдөл үүсгэх, ажиллагааг эхлүүлэх/түр зогсоох/үргэлжлүүлэх/дахин тохируулах болон хурд тохируулах боломжтой. Оролтын шалгалт болон runId ашигласан хамгаалалт нь ажиллагааг илүү найдвартай болгоход чиглэнэ.

Цаашид бодит туршилтын үр дүнг бүрэн бүртгэж, олон хэмжээтэй өгөгдөл дээр гүйцэтгэлийг хэмжих, өөр эрэмбэлэлтийн алгоритмтай харьцуулах болон хэрэглэгчийн интерфейсийн алдааны тохиолдлуудыг өргөтгөх боломжтой.

Ашигласан материал

- F.CSM302 Үйлдлийн систем — Threads & Concurrency бүлэг.
- MDN Web Docs, Web Workers API.
- MDN Web Docs, Worker.postMessage() болон message event.
- Төслийн README.md болон src/ эх кодууд.
- Төслийн tests/test-cases.md.

Хавсралт А. Тестийн жишээ ба кодын тэмдэглэл

Жишээ оролт: 7, 12, 19, 3, 18, 4, 2, 6, 15, 8

Хүлээгдэж буй эрэмбэлэгдсэн үр дүн: 2, 3, 4, 6, 7, 8, 12, 15, 18, 19

Энэ жишээг хөтөч дээр ажиллуулж, бодит үр дүн хүлээгдэж буй үр дүнтэй таарч байгааг шалгасны дараа дэлгэцийн зураг хавсаргана.

[ЭНД нэмэлт кодын хэсэг, туршилтын лог эсвэл дэлгэцийн зураг оруулна.]